INF333 2025-2026 Spring Semester

Fanatics
Önder Akyüz <23401064@ogr.gsu.edu.tr>
Gökay Kayra Bereket <23401065@ogr.gsu.edu.tr>

Homework II
Design Document

Please provide answers inline in a `quote` environment.

1 Preliminaries

Q1: If you have any preliminary comments on your submission, notes for the TAs, or extra credit, please give them here.

> No extra credit. We worked together and split the project. Önder did the thread fields, syscall setup and exec/wait parts. Gökay did the argument passing, file syscalls and deny-write with cleanup.

Q2: Please cite any offline or online sources you consulted while preparing your submission, other than the Pintos documentation, course text, and lecture notes.

> We only used the Pintos documentation and looked at a few Stack Overflow posts about strtok_r usage.

2 Argument Passing

2.1 Data Structures

Q3: Copy here the declaration of each new or changed 'struct' or 'struct' member, global or static variable, 'typedef', or enumeration.

Identify the purpose of each in 25 words or less.

No new structs were added for argument passing. We used two local arrays inside `push_arguments`:

```
char *args[128];  /* argument strings */
char *ptrs[128];  /* their stack addresses */
```

2.2 Algorithms

Q4: Briefly describe how you implemented argument parsing. How do you arrange for the elements of `argv[]` to be in the right order? How do you avoid overflowing the stack page?

We use `strtok_r` to split the command line. The program name goes to `thread_create`, the rest is passed along. After load succeeds, `push_arguments` pushes strings onto the stack right to left, word-aligns, then pushes argv pointers, argv, argc and a fake return address. We cap arguments at 128 to avoid overflow.

2.3 Rationale

Q5: Why does Pintos implement `strtok_r()` but not `strtok()`?

`strtok` uses a global variable internally so it is not thread-safe. In a kernel with multiple threads, two calls at the same time would corrupt each other's state. `strtok_r` takes an explicit save pointer so each caller has its own state.

Q6: In Pintos, the kernel separates commands into an executable name and arguments. In Unix-like systems, the shell does this separation. Identify at least two advantages of the Unix approach.

First, the shell can expand wildcards and variables before the program runs, which the kernel cannot do. Second, the kernel stays simpler since parsing logic lives in userspace and different shells can have different syntax without changing the kernel.

3 System Calls

3.1 Data Structures

Q7: Copy here the declaration of each new or changed 'struct' or 'struct' member, global or static variable, 'typedef', or enumeration. Identify the purpose of each in 25 words or less

We added these fields to `struct thread` in `thread.h` for user programs:

```
int exit_code;                    /* exit code of the process */
struct semaphore sema_load;       /* parent waits for child to load */
bool load_success;                /* child sets this after load() */
tid_t parent_tid;                 /* parent's tid */
struct list child_list;           /* children of this thread */
struct child_status *self_status; /* points to our entry in parent's list */
struct file *exec_file;           /* the executable */
struct file *fd_table[128];       /* fd 0,1 */
int fd_count;                     /* next free fd */
```

In `process.h` we added a struct that the parent stores per child:

```
struct child_status {
  tid_t tid;                  /* child's tid */
  int exit_code;              /* child's exit code */
  bool is_alive;              /* false after child exits */
  bool already_waited;        /* true if wait was called for this child */
  struct semaphore wait_sema; /* parent waits on this */
  struct list_elem elem;      /* element in parent's child_list */
};
```

In `syscall.c` we added one global lock:

```
struct lock filesys_lock;   /* protects all filesystem calls */
```

Q8: Describe how file descriptors are associated with open files. Are file descriptors unique within the entire OS or just within a single process?

Each process has its own `fd_table[128]`. When a file is opened, we store the file pointer at the first free slot from index 2 and return that index as the fd. File descriptors are unique per process, not OS-wide. fd 0 and fd 1 are reserved for stdin and stdout.

Q9: Describe the sequence of events when `lock_release()` is called on a lock that a higher-priority thread is waiting for.

This question is about the priority donation logic from HW1, not something we implemented in this homework.

3.2 Algorithms

Q10: Describe your code for reading and writing user data from the kernel.

Before the kernel uses an user pointer, we check it with `validate_pointer`. It checks for NULL, then `is_user_vaddr`, then `pagedir_get_page`. If any check fails we can call `exit_process(-1)`. For buffers and strings, we have `validate_buffer` and `validate_string` which checks every byte. While reading a 4-byte argument from the stack, we check both the first and the last byte. Thanks to this action, we don't miss a page boundary.

Q11: Suppose a system call causes a full page (4,096 bytes) of data to be copied from user space into the kernel. What is the least and the greatest possible number of inspections of the page table (e.g. calls to `pagedir_get_page()`) that might result? What about for a system call that only copies 2 bytes of data? Is there room for improvement in these numbers, and how much?

For 4096 bytes, the minimum possible number is 1 inspection (if the buffer is page aligned) and the maximum possible number is 2 (if it crosses a page boundary). For 2 bytes, the minimum possible number is also 1 and the maximum possible number is also 2. There is room for improvement. Right now `validate_buffer` can call `pagedir_get_page` once per byte, so for 4096 bytes we make 4096 calls instead of 1 or 2. A better way would be to only check the first byte, the last byte and one byte at the start of each page in between.

Q12: Briefly describe your implementation of the "wait" system call and how it interacts with process termination.

`process_wait` looks for the given tid in the current thread's `child_list`. If it doesn't find it or if `already_waited` is true, it can return -1. Otherwise it sets `already_waited = true` and does `sema_down` on `wait_sema` if the child is still alive. When the child exits; `process_exit` sets the exit code, marks `is_alive = false` and does `sema_up`. The parent wakes up, reads the exit code, frees the struct and returns it.

Q13: Any access to user program memory at a user-specified address can fail due to a bad pointer value. Such accesses must cause the process to be terminated. System calls are fraught with such accesses, e.g. a "write" system call requires reading the system call

number from the user stack, then each of the call's three arguments, then an arbitrary amount of user memory, and any of these can fail at any point. This poses a design and error-handling problem: how do you best avoid obscuring the primary function of code in a morass of error-handling? Furthermore, when an error is detected, how do you ensure that all temporarily allocated resources (locks, buffers, etc.) are freed? In a few paragraphs, describe the strategy or strategies you adopted for managing these issues. Give an example.

> We validate all pointers before doing anything else. `validate_pointer`, `validate_buffer` and `validate_string` are called at the top of each syscall before any lock is acquired. If a pointer is bad, `exit_process(-1)` is called immediately and there is nothing to clean up since no resources were taken yet.

3.3 Synchronization

Q14: The "exec" system call returns -1 if loading the new executable fails, so it cannot return before the new executable has completed loading. How does your code ensure this? How is the load success/failure status passed back to the thread that calls "exec"?

> After `thread_create`, the parent does `sema_down` on its own `sema_load`. The child runs `start_process` and calls `load`. After load returns, the child finds the parent with `get_thread_by_tid(parent_tid)`, writes the result into the parent's `load_success` field and does `sema_up` on `sema_load`. The parent wakes up and checks `load_success`. If that is false, exec returns -1. If that is true, it returns the child's tid. With this way, exec never returns before load is done.

Q15: Consider parent process P with child process C. How do you ensure proper synchronization and avoid race conditions when P calls wait(C) before C exits? After C exits? How do you ensure that all resources are freed in each case? How about when P terminates without waiting, before C exits? After C exits? Are there any special cases?

> If P waits before C exits: P sleeps on `sema_down`. C wakes it up on exit and P frees the `child_status`. If P waits after C exits: `sema_down` returns immediately since C already called `sema_up`. If P exits without waiting: the `child_status` structs stay in memory. C can still write its exit code safely. This is a small leak but keeps things simple.

3.4 Rationale

Q16: Why did you choose to implement access to user memory from the kernel in the way that you did?

> We chose the explicit validation approach (calling `pagedir_get_page` before each access) instead of the page-fault method. The main reason is that is easier to read. Every syscall validates its arguments at the top and if a pointer is bad, we can call `exit_process(-1)`. The downside is that is slower. Because we check every byte. The page-fault method would be faster. But it would mean writing more code in the exception handler. We wanted to keep things simple.

Q17: What advantages or disadvantages can you see to your design for file descriptors?

> Advantage: Array lookup is O(1) and simple to implement. Disadvantage: Fixed size of 128 limits open files per process. Also wastes memory when few files are open.

Q18: The default `tid_t` to `pid_t` mapping is the identity mapping. If you changed it, what advantages are there to your approach?

> We didn't change the default. `pid_t` is the same value as `tid_t`. This works. Because in Pintos, every users process is just one thread, so the two ids always match. The advantage is that the code stays simple. We don't need an extra table to map between them.

4 Survey Questions

Answering these questions is optional, but it will help us improve the course in future quarters. Feel free to tell us anything you want–these questions are just to spur your thoughts. You may also choose to respond anonymously in the course evaluations at the end of the quarter.

Q1: In your opinion, was this assignment, or any one of the three problems in it, too easy or too hard? Did it take too long or too little time?

> The assignment was challenging but also doable. The exec/wait part with synchronization took us the most time. Because we had to think carefully about race conditions.

Q2: Did you find that working on a particular part of the assignment gave you greater insight into some aspect of OS design?

Yes, working on exec and wait, gave us a much better understanding of how parent and child processes communicate in a real OS.

Q3: Is there some particular fact or hint we should give students in future quarters to help them solve the problems? Conversely, did you find any of our guidance to be misleading?

A clear stack diagram for argument passing earlier in the docs would have helped. We spent too much time debugging the layout.

Q4: Do you have any suggestions for us to more effectively assist students, either for future semesters or the remaining projects?

Sometimes, the difficulty of the homework feels a bit heavy compared to our level of background knowledge and we get stuck for a long time on small details. It would be helpful if this could be balanced a bit.

Q5: Any other comments?

Good assignment overall, we learned a lot.